Transmission Control Protocol

(TCP)

Outline

TCP Services

TCP Features

Segment

TCP Connection

Windows in TCP

Flow Control

Error Control

Congestion Control

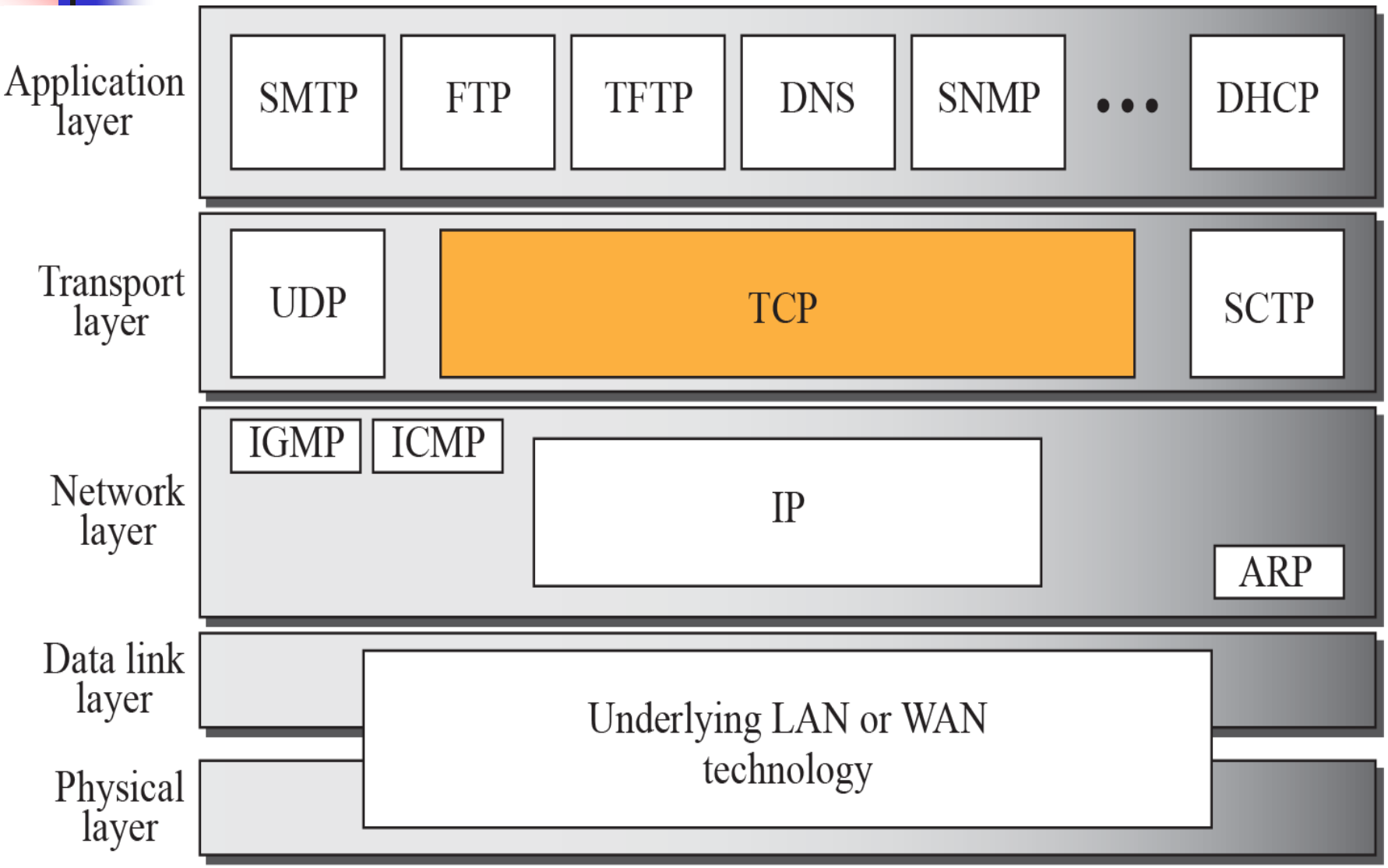
Options

TCP SERVICES

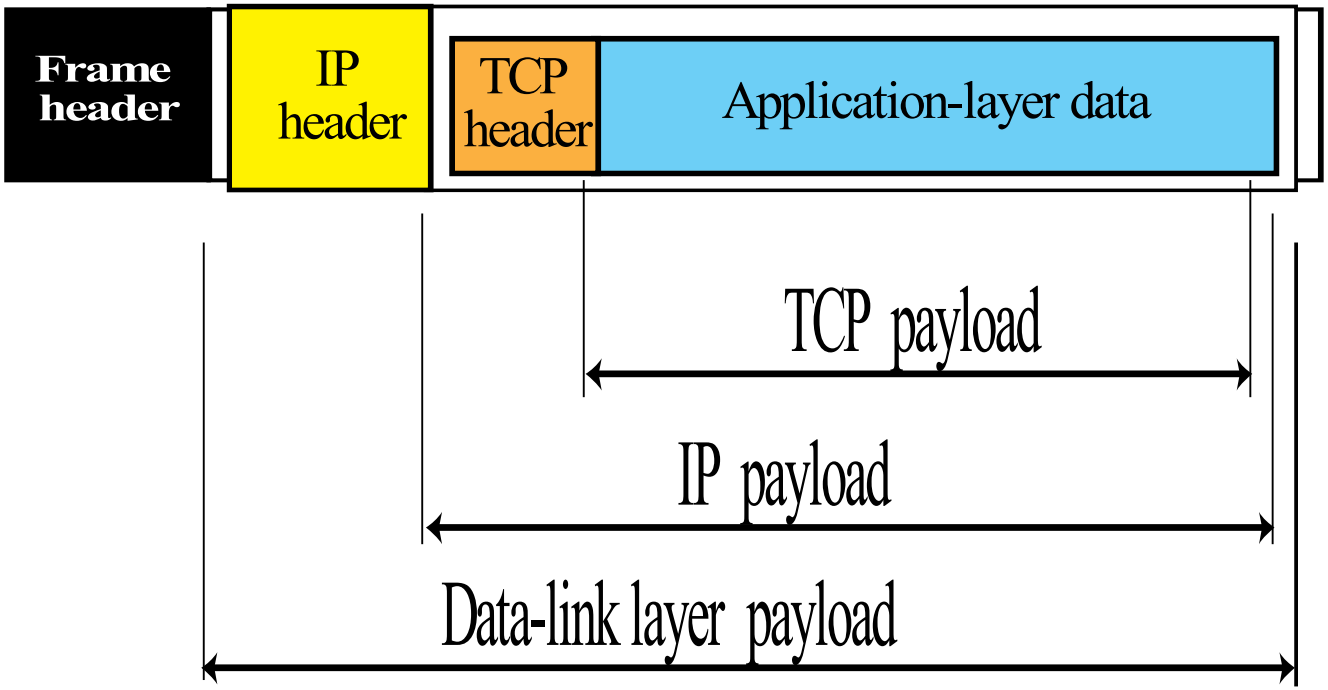
TCP lies between the application layer and the network layer,

It serves as the intermediary between the application programs and the network operations.

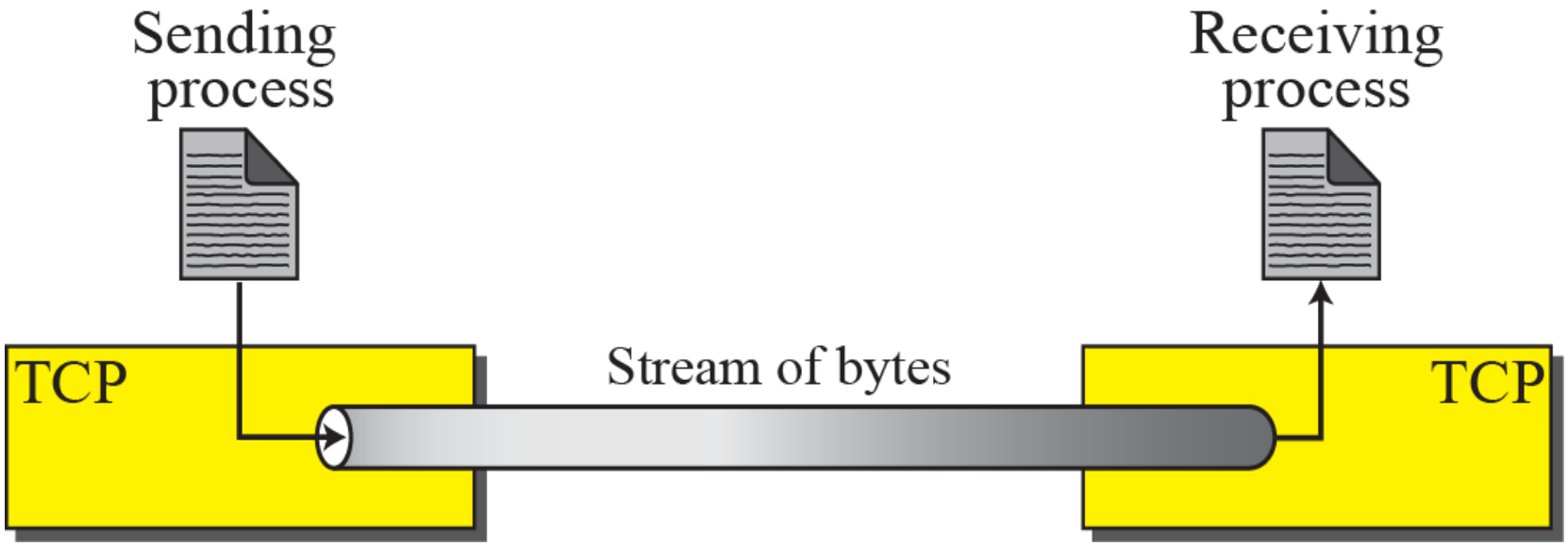
TCP/IP protocol suite



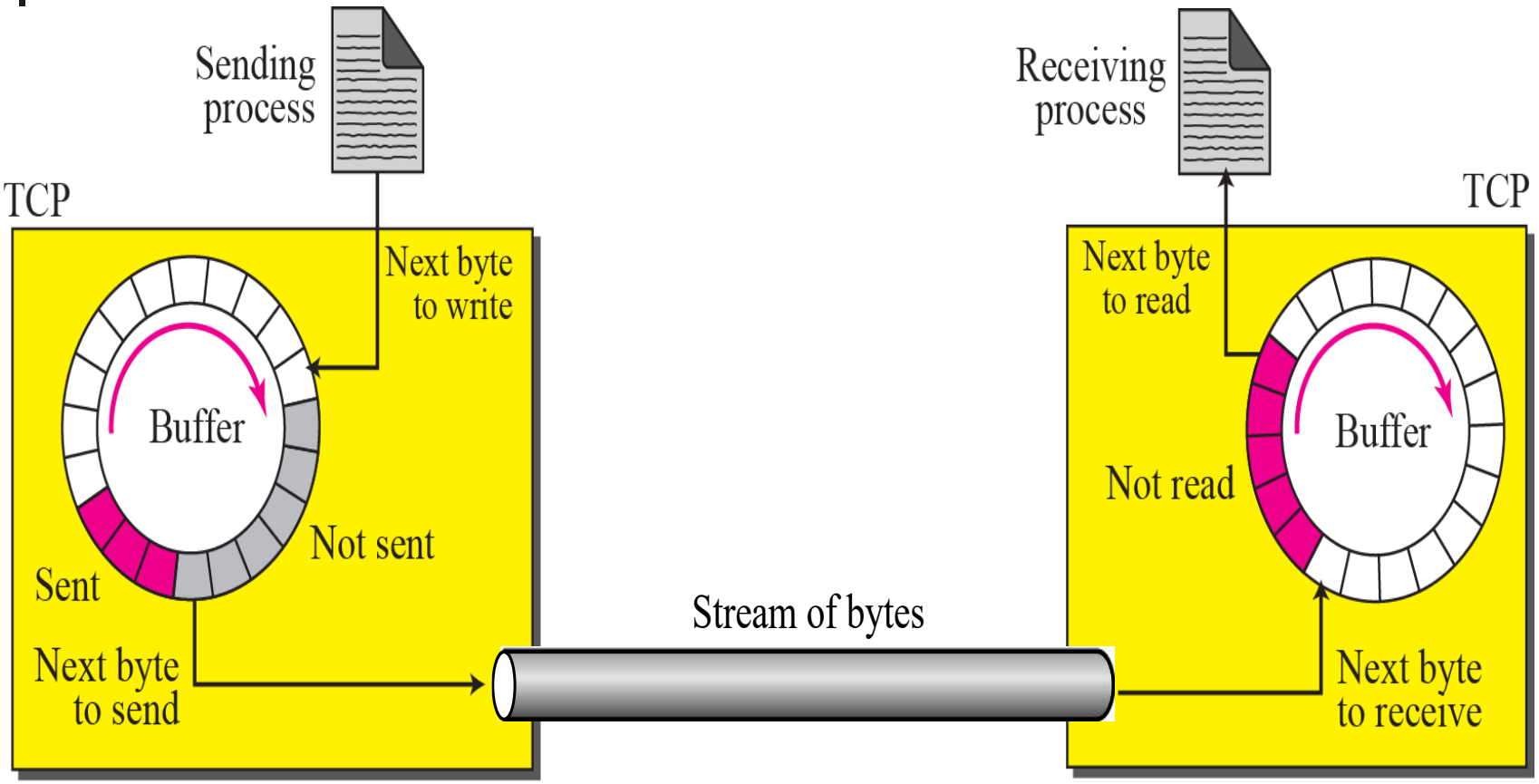
Encapsulation



Stream delivery Service



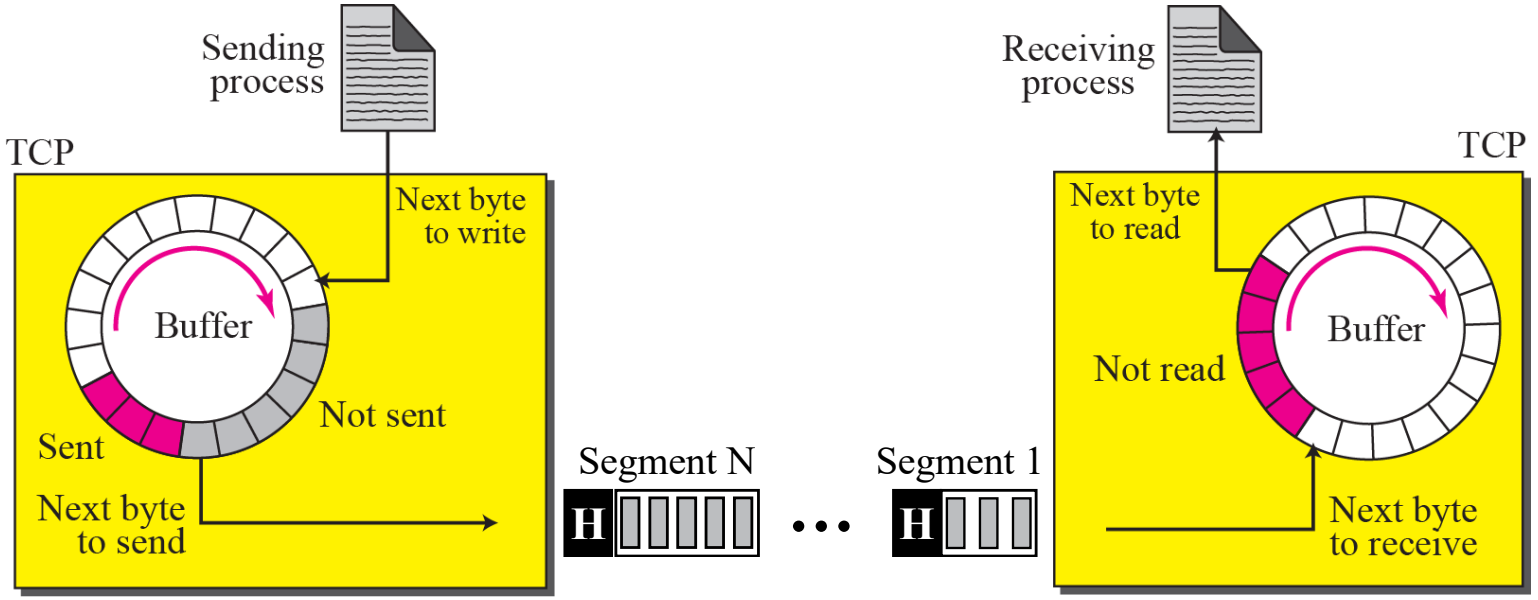
Sending and receiving buffers



TCP segments

The bytes of data being transferred in each connection are numbered by TCP.

The numbering starts with an arbitrarily generated number.



Example

Suppose a TCP connection is transferring a file of 5,000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1,000 bytes?

The following shows the sequence number for each segment:

Segment 1	→	Sequence Number:	10,001	Range:	10,001	to	11,000
Segment 2	→	Sequence Number:	11,001	Range:	11,001	to	12,000
Segment 3	→	Sequence Number:	12,001	Range:	12,001	to	13,000
Segment 4	→	Sequence Number:	13,001	Range:	13,001	to	14,000
Segment 5	→	Sequence Number:	14,001	Range:	14,001	to	15,000

sequence number

The value in the sequence number field of a segment defines the number assigned to the first data byte contained in that segment.

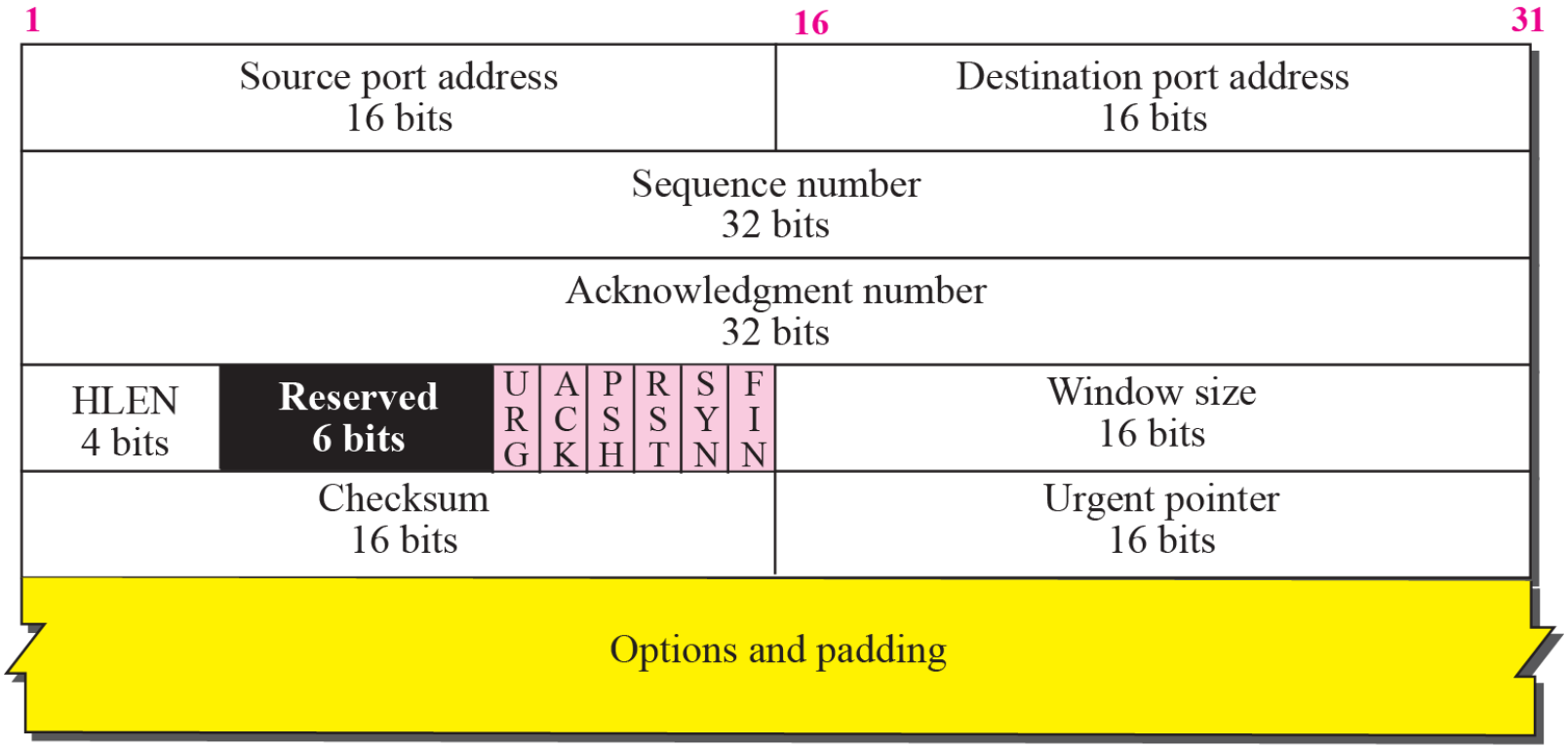
acknowledgment number

The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive.

The acknowledgment number is cumulative.

A packet in TCP is called a segment.

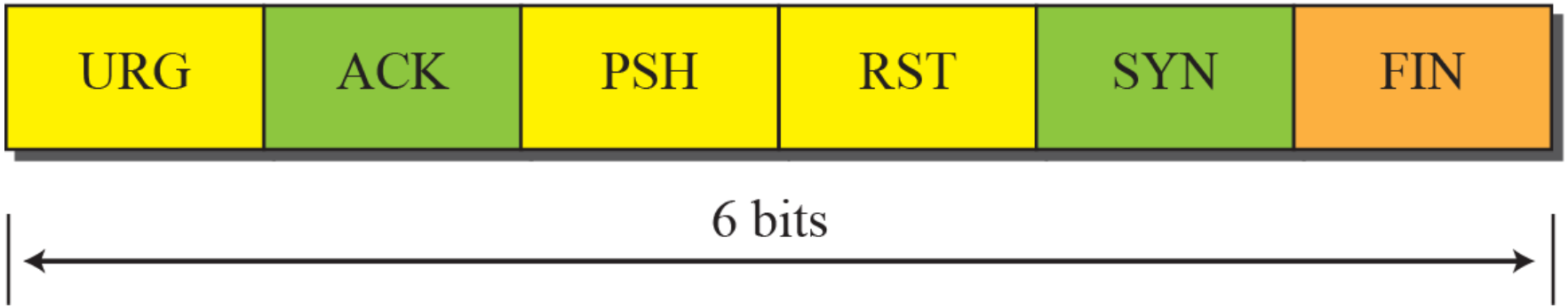
a. Segment



b. Header

Control field

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push
RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection



TCP CONNECTION

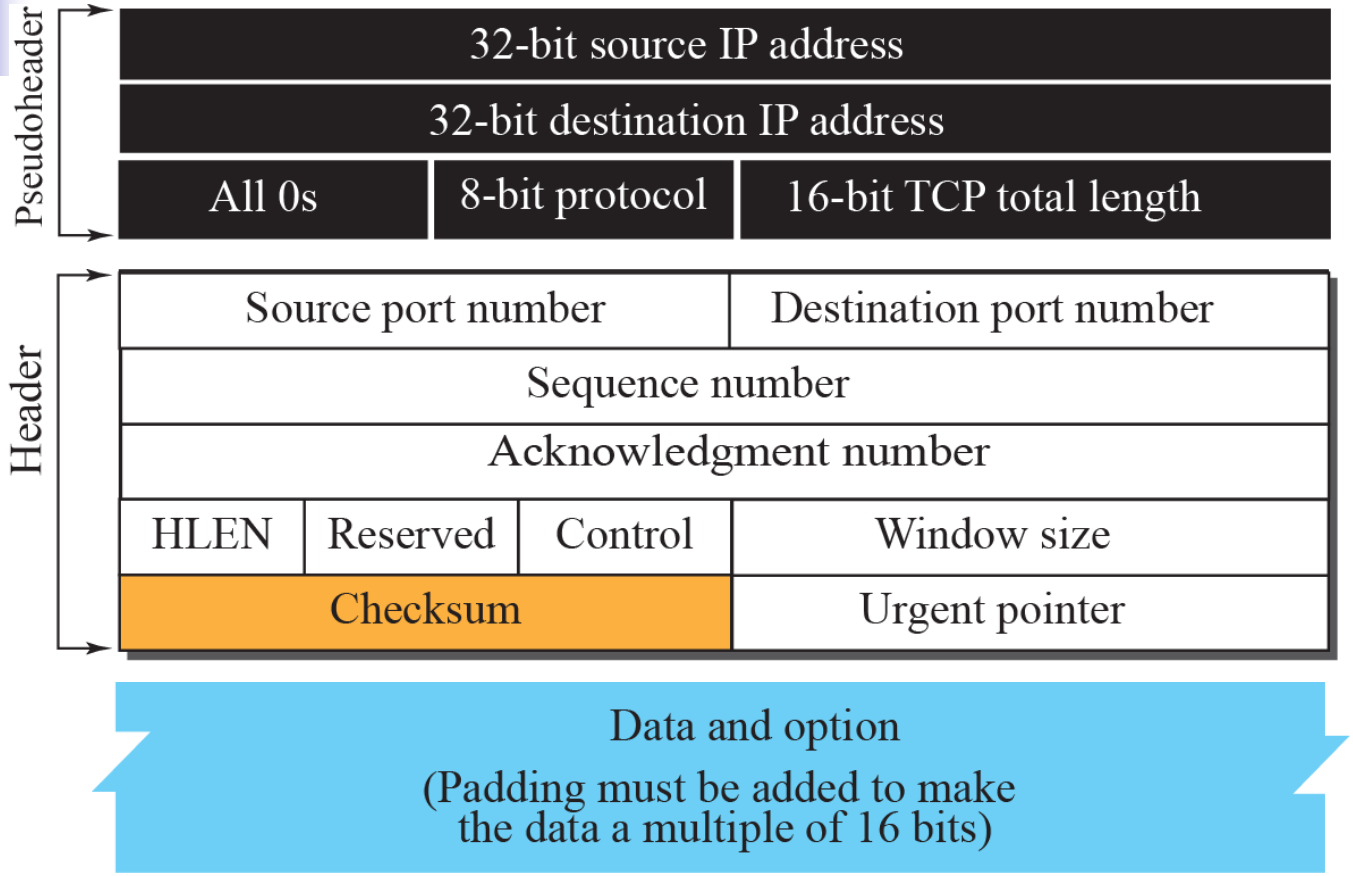
TCP is connection-oriented.

It establishes a virtual path between the source and destination. All of the segments belonging to a message are then sent over this virtual path.

How TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented?

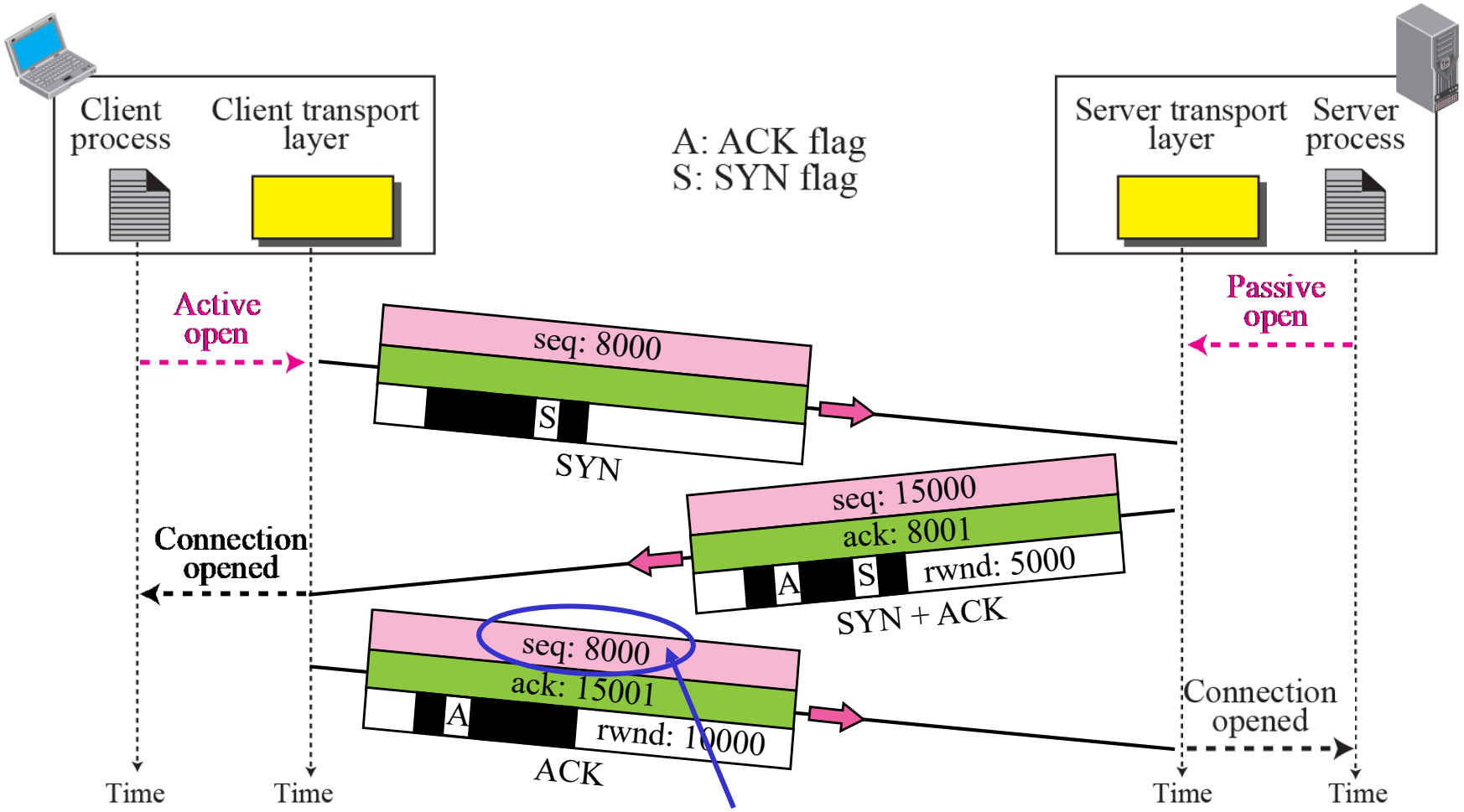
A TCP connection is virtual, not physical. TCP operates at a higher level. TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself. If a segment is lost or corrupted, it is retransmitted.

Pseudoheader added to the TCP segment



The use of the checksum in TCP is mandatory.

Connection establishment using three-way handshake

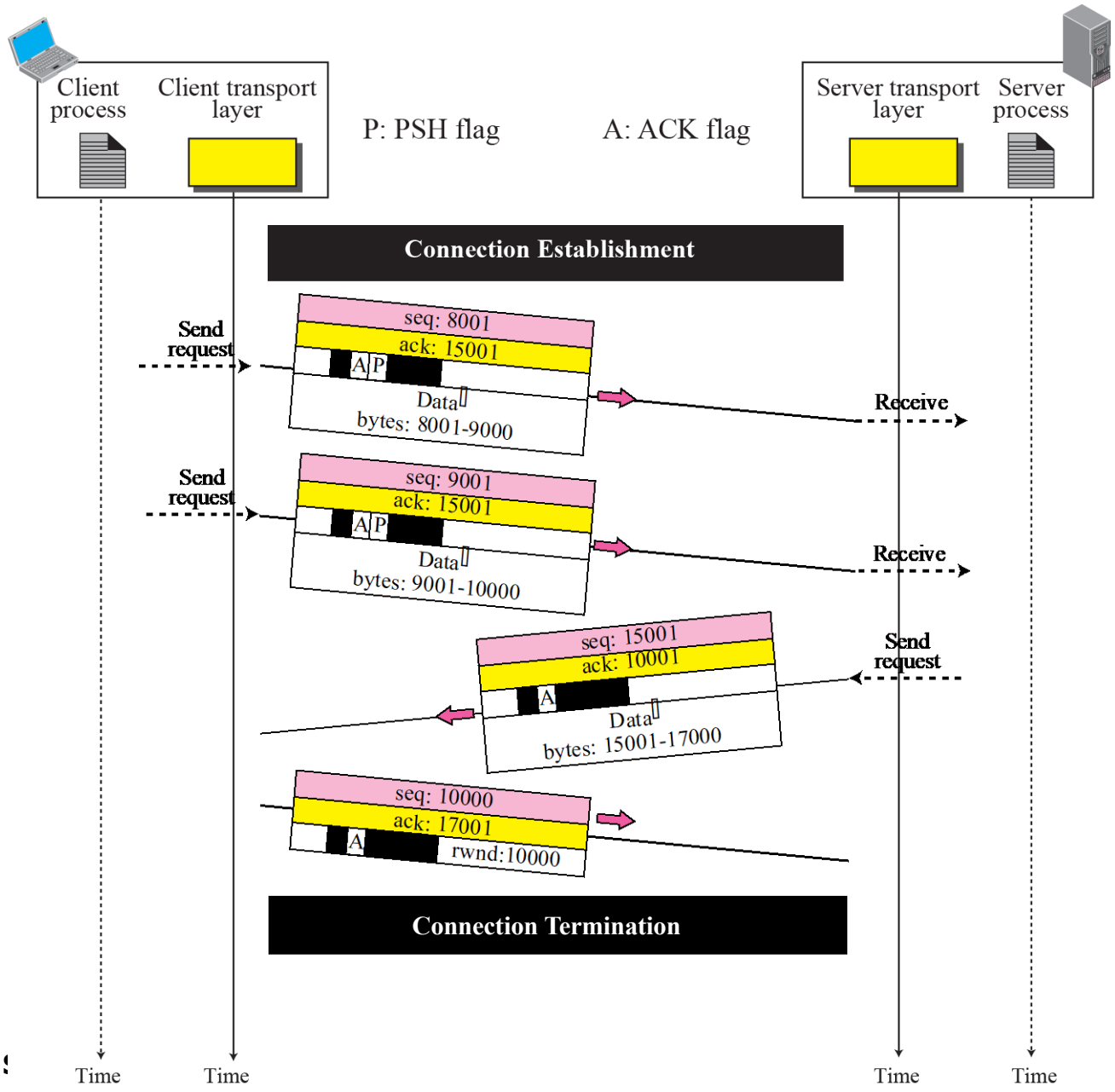


A SYN segment cannot carry data, but it consumes one sequence number.

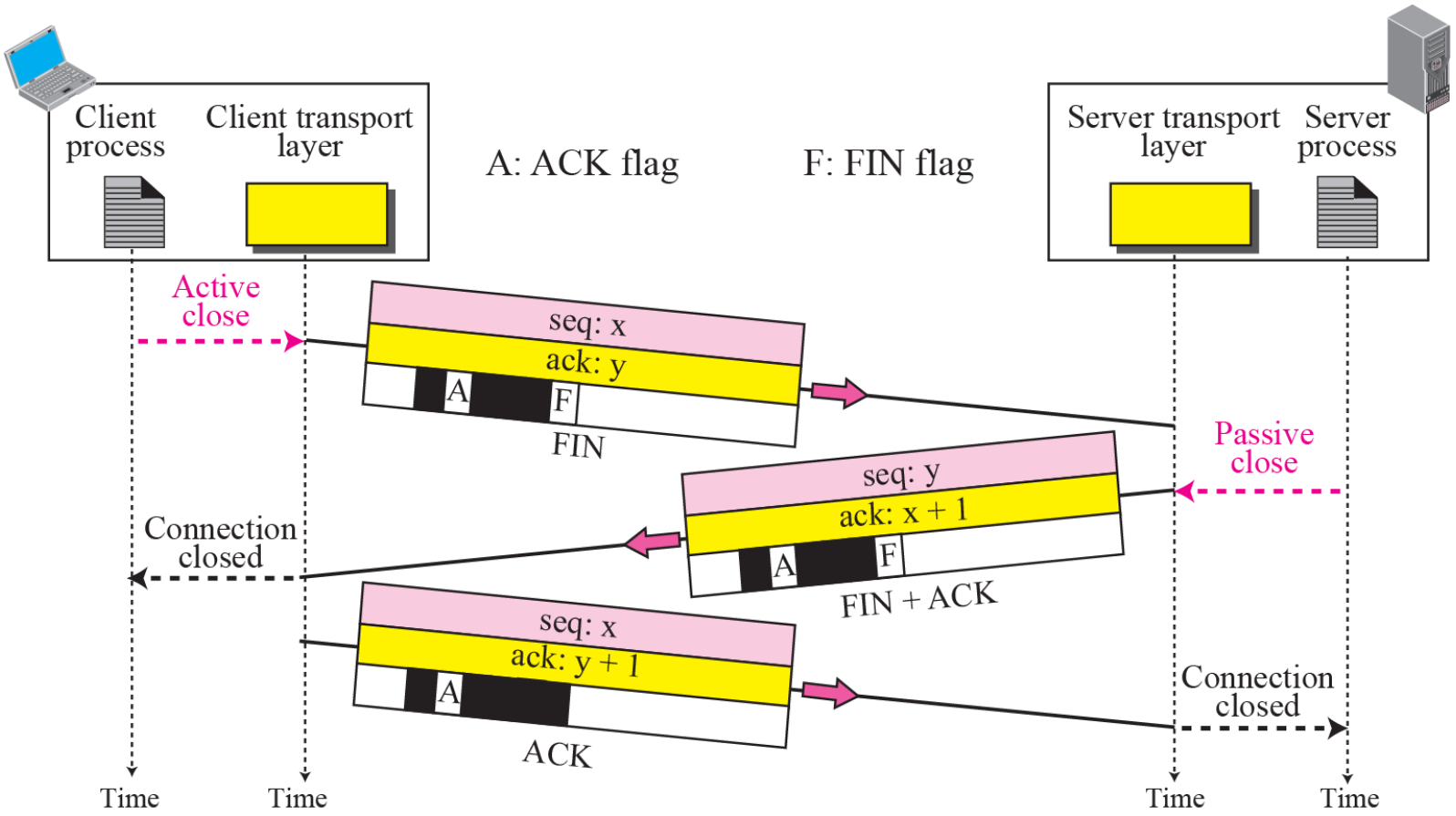
A SYN + ACK segment cannot carry data, but does consume one sequence number.

An ACK segment, if carrying no data, consumes no sequence number.

Data Transfer



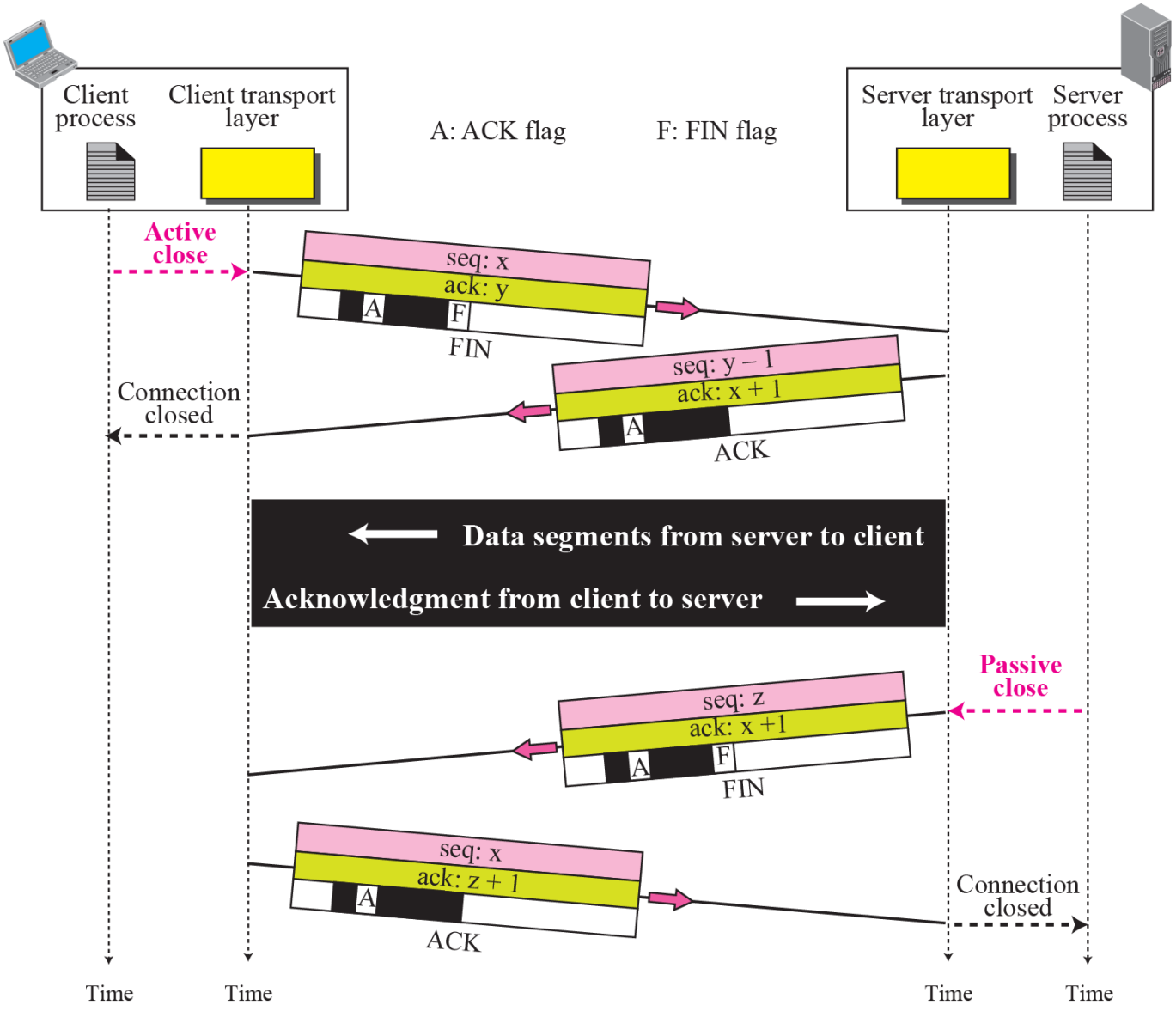
Connection termination using three-way handshake



The FIN segment consumes one sequence number if it does not carry data.

FIN + ACK segment consumes one sequence number if it does not carry data.

Half-Close


Client process
Client transport layer
A: ACK flag
F: FIN flag
Server transport layer
Server process
Active close
seq: x
ack: y
A
F
FIN
seq: y – 1
ack: x + 1
A
ACK
Connection closed
Data segments from server to client
Acknowledgment from client to server
Passive close
seq: z
ack: x +1
A
F
FIN
seq: x
ack: z + 1
A
ACK
Connection closed
Time
Time
Time
Time

WINDOWS IN TCP

TCP uses two windows for each direction of data transfer

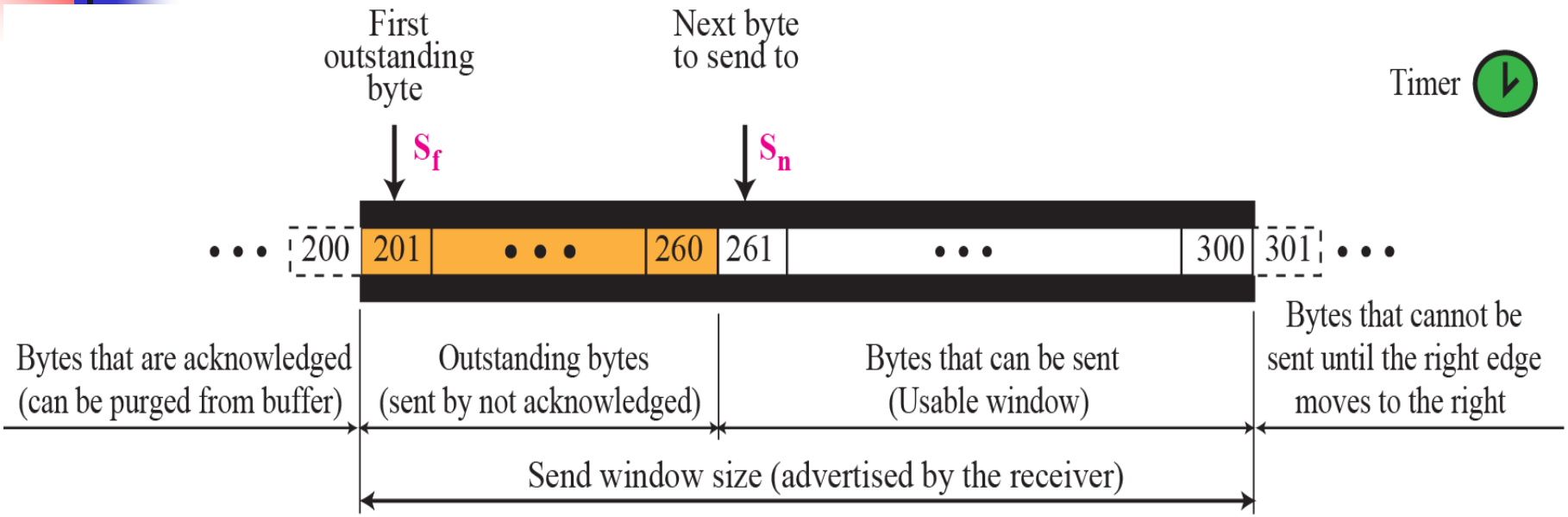
(send window and receive window)

(i.e. four windows for a bidirectional communication)

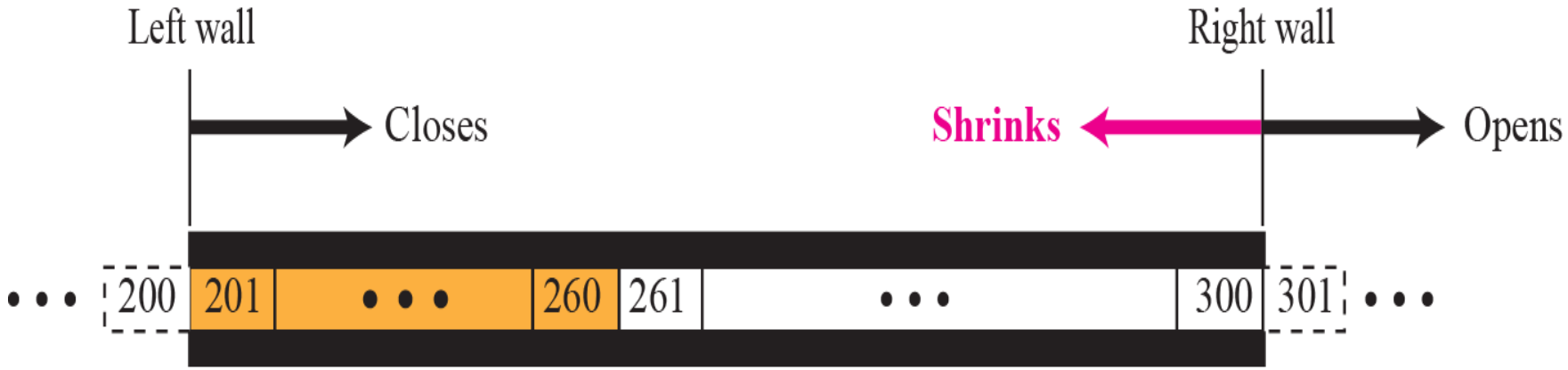
To make the discussion simple, we make an assumption that communication is only unidirectional;

the bidirectional communication can be inferred using two unidirectional communications with piggybacking.

Send window in TCP



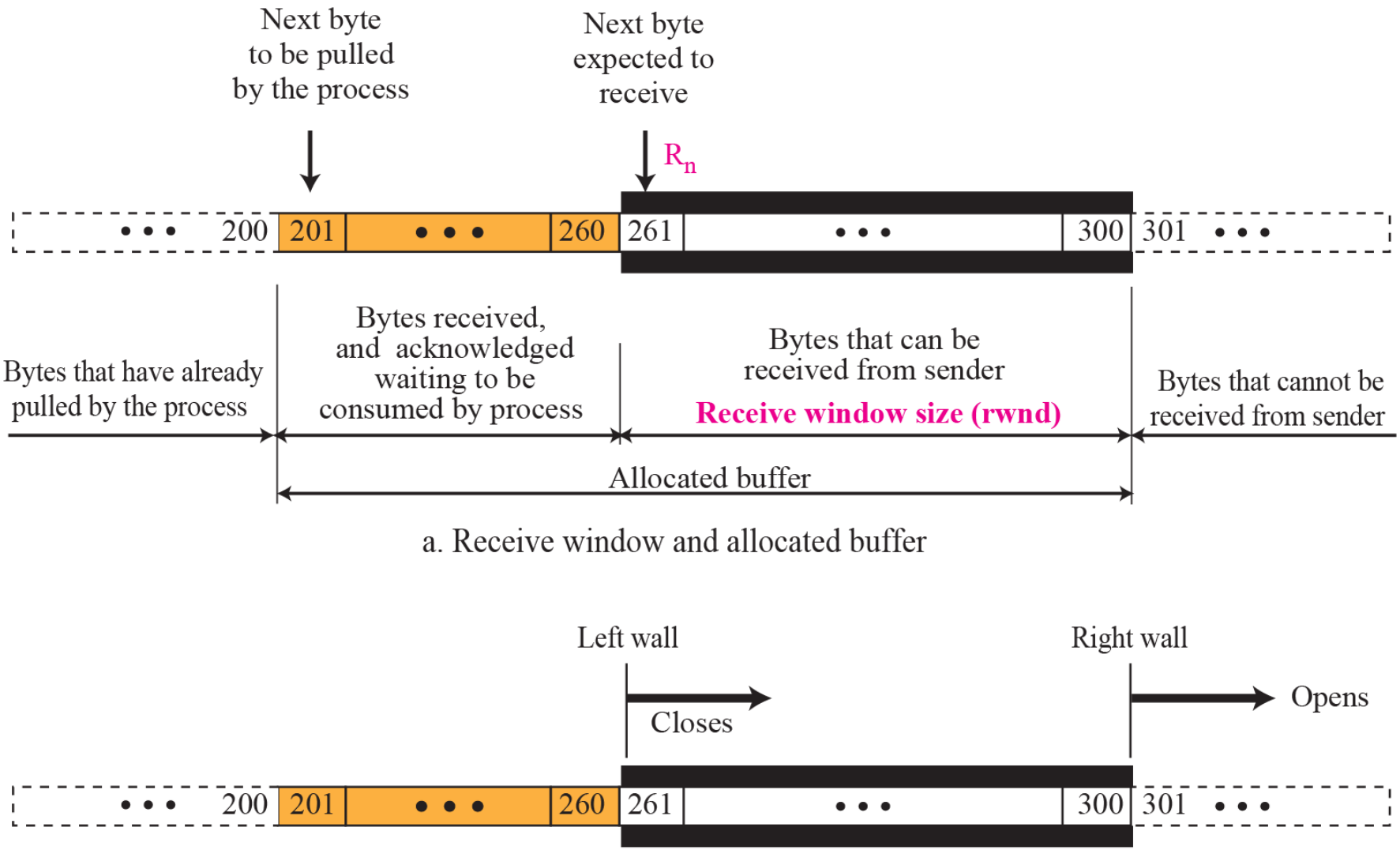
a. Send window



b. Opening, closing, and shrinking send window

Receive window in TCP

rwnd= buffer size – number of bytes waiting to be pulled



a. Receive window and allocated buffer

b. Opening and closing of receive window

FLOW CONTROL

Flow control balances the rate a producer creates data with the rate a consumer can use the data.

TCP separates flow control from error control.

We temporarily assume that the logical channel between the sending and receiving TCP is error-free.

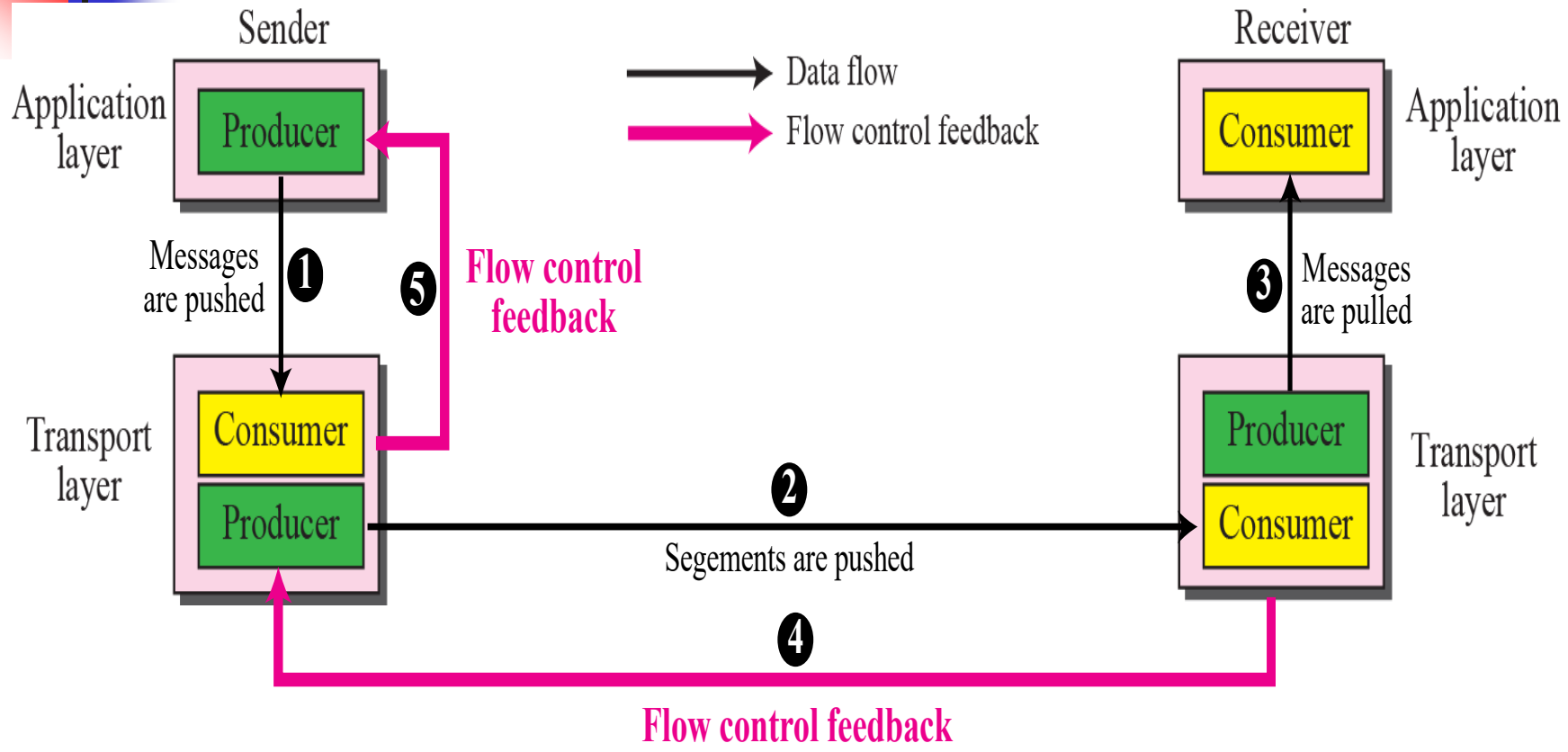
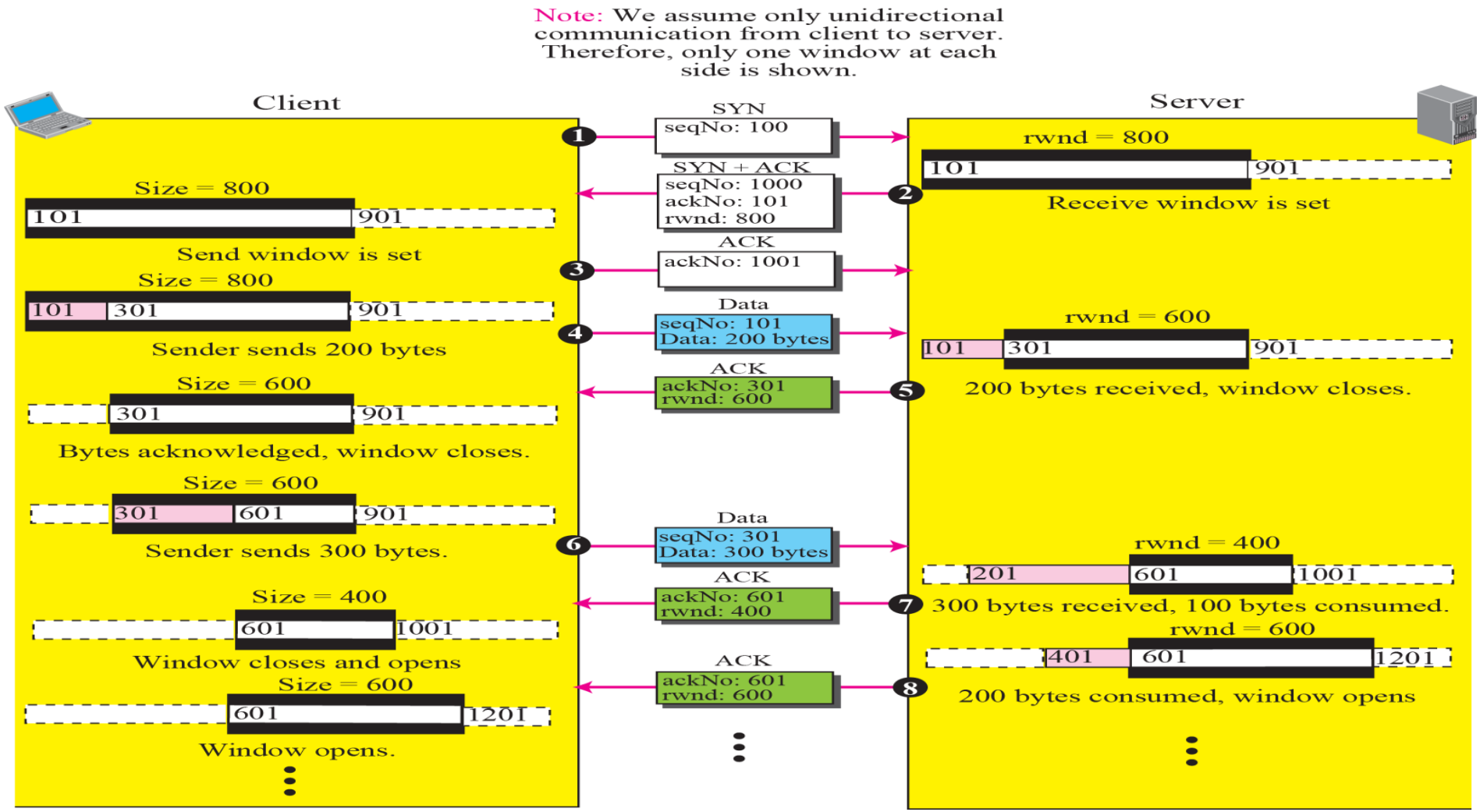


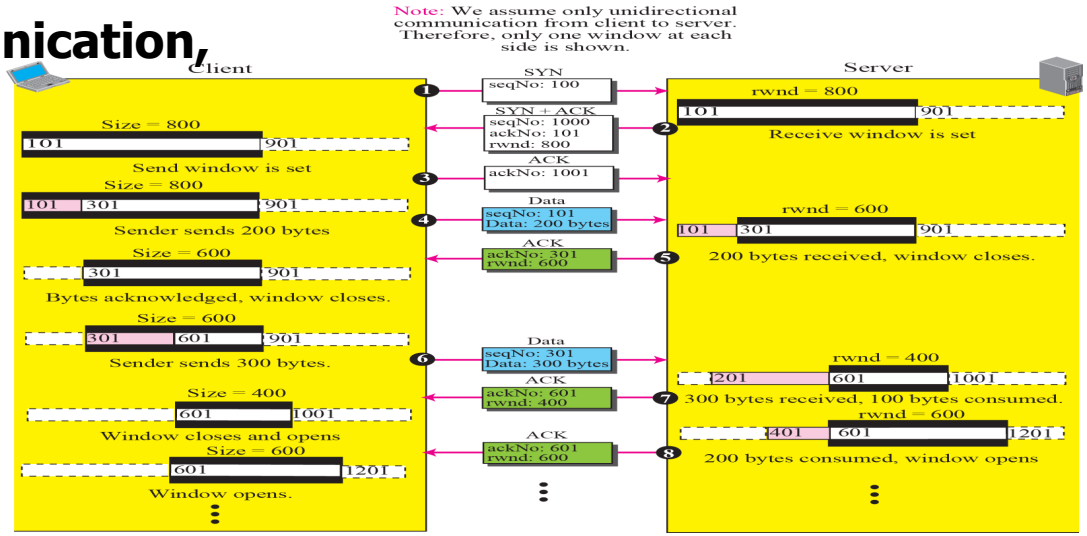
Figure shows

- unidirectional data transfer between a sender and a receiver;
- bidirectional data transfer can be deduced from unidirectional one

An example of flow control



Assuming uni-directional communication, Represent the following TCP transmission



1. Seq. No =100
2. Ack= 101 rwnd =800
3. Send 200 B
4. Ack = 301 rwnd = 600
5. Send 300 B
6. 100 B consumed
7. 200 B consumed
8. 200 B sent
9. 100 B consumed, ack sent
10. 100 B sent

Ques

A window holds bytes 3000 to 6000 and the next byte to be sent is 5000. How will the start of the window, end of the window and next byte to be sent represented if a single ACK with the acknowledgment number 3300 and window size advertisement of 4200 is received by the sender.

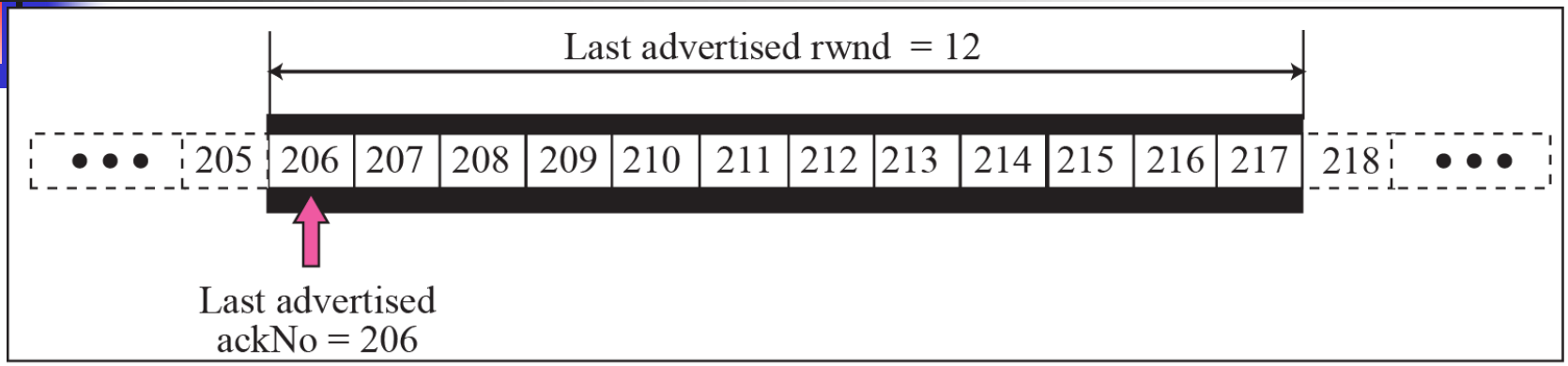
Start of the Window ?

End of the window?

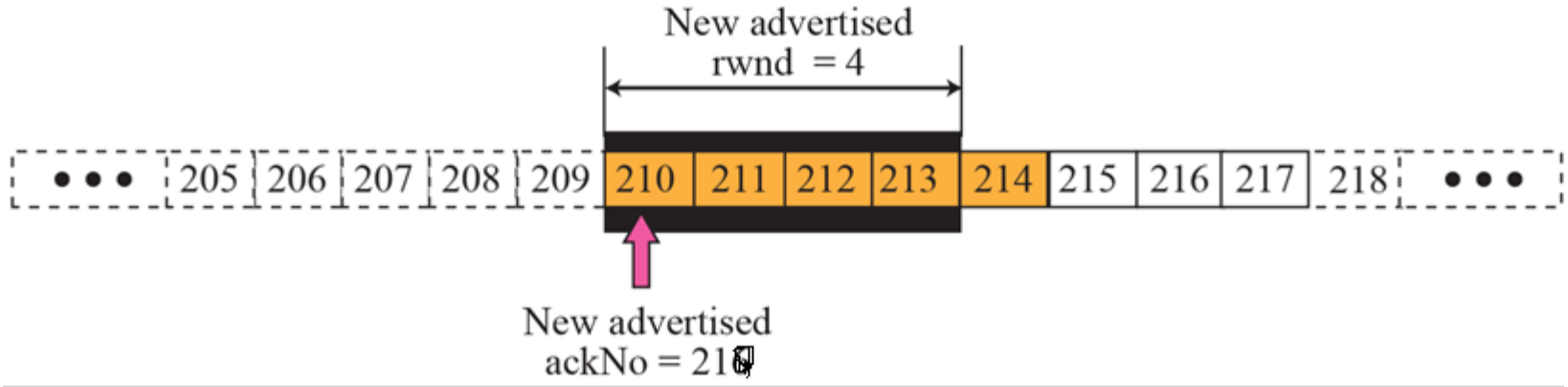
Next byte to sent ?

(3300,7499,5000)

Shrinking of Windows



a. The window after the last advertisement



b. The window after the new advertisement; window has shrunk

The new advertisement, however, defines the new value of *rwnd* as 4, in which **210 + 4 < 206 + 12**.

(byte 214 which has been already sent is outside the window)

When the send window shrinks, it may creates a problem:

Following relations can need to maintained by receiver to prevent shrinking

New ack number + new rwnd > = Last ack number + Last rwnd

One way to prevent this situation is:
let the receiver postpone its feedback until enough buffer locations are available in its window. (to meet the above relation)

In other words, the receiver should wait until more bytes are consumed by its process.

Silly Window Syndrome

Sending data in very small segments

I. Syndrome created by the Sender

- Sending application program that creates data slowly (e.g. 1 byte at a time)

Prevention

- Wait and collect data to send in a larger block

- How long should the sending TCP wait?
- Solution: Nagle's algorithm

Nagle's algorithm

1. The sending TCP sends the first piece of data it receives from the sending application program even if it is only 1 byte.
2. After sending the first segment, the sending TCP accumulates data in the output buffer and waits until either the receiving TCP sends an acknowledgment or until enough data has accumulated to fill a maximum-size segment. At this time, the sending TCP can send the segment.
3. Step 2 is repeated for the rest of the transmission. Segment 3 is sent immediately if an acknowledgment is received for segment 2, or if enough data have accumulated to fill a maximum-size segment.

Nagle's algorithm takes into account
(1) the speed of the application program that creates the data, and
(2) the speed of the network that transports the data

Observation

1. If application program faster than network, segments are larger
2. If application program slower than network, segments are smaller

Silly Window Syndrome

Syndrome created by the Receiver

- Receiving application program consumes data slowly (e.g. 1 byte at a time)

The receiving TCP announces a window size of 1 byte. The sending TCP sends only 1 byte...

Two Solution Proposed

- I: Clark's solution
- II: Delayed Acknowledgement

Clark's Approach

- Sending an ACK as soon as data arrives but announcing a window size of zero until there is enough space to accommodate a segment of max. size or until half of the buffer is empty

Silly Window Syndrome

Solution 2: **Delayed Acknowledgement**

- The receiver waits until there is decent amount of space in its incoming buffer before acknowledging the arrived segments

- Advantage
- The delayed acknowledgement prevents the sending TCP from sliding its window.
- It also reduces traffic.

- Disadvantage:
- it may force the sender to retransmit the unacknowledged segments

- TCP balances the advantage & disadvantage and defines that the acknowledgement should not be delayed by more than 500ms